

UNIVERSIDAD POLITÉCNICA DE MADRID

GRADO EN INGENIERÍA BIOMÉDICA /
GRADO EN INGENIERÍA DE TECNOLOGÍAS Y
SERVICIOS DE TELECOMUNICACIÓN

ALGORITMOS Y ESTRUCTURAS DE DATOS /
ANÁLISIS Y DISEÑO DE SOFTWARE

Ejercicios de algoritmos y estructuras de datos

Profesor:

Rodrigo García Carmona

rodrigo.garcia@upm.es

Versión 1.2



1. Introducción

A continuación encontrará los enunciados de una serie de ejercicios que le servirán para practicar. Para cada ejercicio, aunque no se especifique, no deje de tener en cuenta que:

- Debe considerar los casos límite o problemáticos como arrays vacíos, valores `null`, índices fuera de rango...
- El código debe ser eficiente: si es posible evitar un bucle, ahorrar vueltas dentro de un bucle o reducir el número de líneas a ejecutar, es mejor hacerlo.
- Lo programado debe ser legible, intentando que el código sea fácil de entender y acompañándolo de comentarios completos pero concisos.
- Puede organizar su código en paquetes y clases, así como reutilizar métodos ya creados. Un buen método debería caber en una única pantalla.

Además, no deje de probar su código. No se limite a ver que compila o que funciona con una entrada de datos trivial.

2. Ejercicios de recursividad

En la mayoría de los ejercicios de esta sección se proporciona una estructura de datos en Java. En todos los casos debe implementar un método recursivo estático.

2.1. Sistema de archivos

Se ha diseñado una estructura para modelar un sistema de archivos simple:

```
// Representa un archivo con un tamaño específico
public class Archivo {
    private String nombre;
    private double pesoMB; // Tamaño en Megabytes

    public Archivo(String nombre, double peso) { ... }
    public double getPesoMB() { return this.pesoMB; }
}

// Representa una carpeta que puede contener archivos y otras subcarpetas
public class Carpeta {
    private String nombre;
    private List<Archivo> archivos; // Archivos directos en esta carpeta
    private List<Carpeta> subcarpetas; // Carpetas dentro de esta carpeta

    public Carpeta(String nombre) { ... }
    public List<Archivo> getArchivos() { return this.archivos; }
    public List<Carpeta> getSubcarpetas() { return this.subcarpetas; }
}
```

Implemente el método recursivo `calcularPesoTotal`: Este método debe devolver la suma total del peso (en MB) de una `Carpeta` y todo su contenido (archivos directos y archivos dentro de subcarpetas, recursivamente).

```
public static double calcularPesoTotal(Carpeta inicio) {}
```

2.2. Jerarquía de una empresa

Se utiliza una estructura de árbol para representar el organigrama de una empresa:

```
public class Empleado {
    private String nombre;
    private double salario;
    private List<Empleado> subordinados; // Lista de empleados a su cargo

    public Empleado(String nombre, double salario) { ... }
    public double getSalario() { return this.salario; }
    public List<Empleado> getSubordinados() { return this.subordinados; }
}
```

Implemente el método recursivo `presupuestoEquipo`: Este método debe calcular el coste total de salarios de un equipo, incluyendo el salario del `Empleado` pasado como parámetro y el de todos sus subordinados (directos e indirectos).

```
public static double presupuestoEquipo(Empleado jefe) {}
```

2.3. Árbol de habilidades de un videojuego

Un videojuego organiza las habilidades de sus personajes en un árbol de dependencias, de tal manera que para poder desbloquear una habilidad primero se deben tener las anteriores, además de una serie de puntos.

```
public class Habilidad {
    private String id;
    private int costePuntos; // Coste para desbloquear esta habilidad
    private List<Habilidad> desbloqueables; // Habilidades que dependen de esta

    public int getCoste() { return this.costePuntos; }
    public List<Habilidad> getDesbloqueables() { return this.desbloqueables; }
}
```

Implemente el método recursivo `costeRamaCompleta`, que calcula cuántos puntos totales hacen falta para desbloquear la habilidad raíz y todas las habilidades que descienden de ella en el árbol (es decir, la rama completa).

```
public static int costeRamaCompleta(Habilidad raiz) {}
```

2.4. Página web

Se modela una página web como un árbol de elementos, siguiendo el modelo DOM (*Document Object Model*), el más común:

```
public class ElementoHTML {
    private String tag; // Ejemplo: "div", "p", "img"
    private List<ElementoHTML> hijos;

    public String getTag() { return this.tag; }
    public List<ElementoHTML> getHijos() { return this.hijos; }
}
```

Implemente el método recursivo `contarEtiquetas`. Este método devuelve un entero indicando cuántas veces aparece una etiqueta específica (`tagBuscado`) dentro del árbol que comienza en `elemento`.

```
public static int contarEtiquetas(ElementoHTML elemento, String tagBuscado) {}
```

2.5. Cadena de suministros

Un producto se fabrica a partir de piezas, que a su vez pueden estar formadas por otras piezas.

```
public class Pieza {
    private String nombre;
    private boolean esDefectuosa;
    private List<Pieza> componentes; // Piezas necesarias para montar esta

    public boolean isDefectuosa() { return this.esDefectuosa; }
    public List<Pieza> getComponentes() { return this.componentes; }
}
```

Implemente el método recursivo `contieneDefectos`: Devuelve `true` si la `piezaPrincipal` o cualquiera de sus componentes (en cualquier nivel de profundidad) está marcado como defectuoso. Si todas las piezas están bien, devuelve `false`.

```
public static boolean contieneDefectos(Pieza piezaPrincipal) {}
```

2.6. Palíndromo recursivo

Escriba un método recursivo `esPalindromo(String texto)` que devuelva `true` si la cadena `texto` es un palíndromo (se lee igual al derecho y al revés) y `false` en caso contrario. Asuma que el texto no contiene espacios en blanco y que todos los caracteres están en minúsculas.

```
public static boolean esPalindromo(String texto) {}
```

Si necesita ayuda: El caso base es una cadena de longitud 0 o 1. El caso recursivo compara el primer y último carácter y llama a la función con la subcadena interior.

2.7. Validación de un árbol binario

Dado un árbol binario, valide si cumple con la propiedad de un BST (*Binary Search Tree*, árbol binario de búsqueda): en cada nodo, todos los valores de los nodos del subárbol izquierdo son menores al del nodo, y todos los del derecho son mayores. Implemente la función auxiliar recursiva `esBSTAux`, que mantiene el rango de valores válidos mediante los nodos antecesores que lo acotan: `min` por debajo y `max` por arriba. Un límite a `null` indica que ese lado no está acotado, como ocurre en la raíz.

```
public class Nodo { int valor; Nodo izq, der; }

public static boolean esBST(Nodo nodo) {
    return esBSTAux(nodo, null, null);
}

private static boolean esBSTAux(Nodo nodo, Nodo min, Nodo max) {}
```

2.8. Invertir una lista enlazada

Dada una lista enlazada simple (cada nodo tiene una referencia al siguiente, pero no al anterior), definida por la clase `NodoLista`, implemente un método recursivo que devuelva el nodo cabecera de otra lista del mismo tipo, con los mismos nodos, pero en orden inverso. El caso base ya se le proporciona:

```
public class NodoLista { int dato; NodoLista siguiente; }

public static NodoLista invertirRecursivo(NodoLista actual) {
    // Caso base: lista vacía o único elemento
    if (actual == null || actual.siguiente == null) {
        return actual;
    }
    // Paso recursivo:
    // TODO: Complete este código
}
```

2.9. ¿Tengo el cambio justo?

Implemente un método recursivo que permita calcular de cuántas formas distintas se puede sumar la cantidad **objetivo** usando monedas de valores 1, 2 y 5. Por ejemplo, para **objetivo** = 3 hay 2 formas: (1+1+1) y (1+2).

A continuación se le proporciona la cabecera del método a implementar. En ella, **monedas** es un array de tamaño 3 que contiene:

- En la posición 0, el número de monedas de 1.
- En la posición 1, el número de monedas de 2.
- En la posición 2, el número de monedas de 5.

```
public static int contarFormasCambio(int objetivo, int[] monedas) {}
```

Pista: Los casos base son:

- **objetivo** == 0 (se encontró una forma),
- **objetivo** < 0 (se pasó) y
- no quedan monedas.

2.10. Fusión de listas ordenadas

La fusión de dos listas ordenadas es la base del algoritmo *Merge sort*, uno de los algoritmos que se vio en clase. Dadas dos listas enlazadas simples **l1** y **l2** como las del ejercicio 2.8, que ya están ordenadas de menor a mayor, implemente un método recursivo que devuelva una única lista ordenada fusionando ambas.

```
public static NodoLista fusionarListas(NodoLista l1, NodoLista l2) {}
```

3. Complejidad y algoritmos

Analice el código proporcionado y responda a las preguntas sobre trazado, conteo de operaciones y complejidad en notación $\mathcal{O}$.

3.1. Variable contador

Observe el comportamiento de la variable **contador** en el siguiente código:

```
int contador = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < i; j++) {
        contador++;
    }
}
```

- ¿Cuál será su valor final en función de N ?
- ¿Cuál es la complejidad computacional en notación $\mathcal{O}$ del método?

3.2. Método misterioso (I)

Examine el siguiente método:

```
public static void procesar(int[] datos) {
    int n = datos.length;
    for (int i = 1; i < n; i++) {
        int clave = datos[i];
        int j = i - 1;
        while (j >= 0 && datos[j] > clave) {
            datos[j + 1] = datos[j];
            j = j - 1;
        }
        datos[j + 1] = clave;
    }
}
```

- Si `datos = {5, 2, 4, 6, 1, 3}`, ¿qué contendrá el array tras finalizar la iteración en la que `i=2`?
- ¿Qué algoritmo implementa este código?
- ¿Cuál es la complejidad computacional en el peor caso y en el mejor caso? Justifique su respuesta.

3.3. Bucles anidados (I)

Examine el siguiente fragmento de código:

```
public static long calcular(int n) {
    long contador = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            contador++;
        }
    }
    return contador;
}
```

- Si `n=5`, ¿cuál es el valor final de `contador`?
- ¿Cuántas veces se ejecuta la instrucción `contador++` en función de `n`? Exprese el resultado como una suma aritmética.
- Deduzca la complejidad $\mathcal{O}$ del algoritmo.

3.4. Búsqueda binaria modificada

Examine el siguiente método recursivo que opera sobre un array **ordenado**:

```
public static int busqueda(int[] arr, int inicio, int fin, int x) {
    if (fin < inicio) return -1;
    int medio = inicio + (fin - inicio) / 2;
    if (arr[medio] == x) return medio;
    if (arr[medio] > x)
        return busqueda(arr, inicio, medio - 1, x);
    return busqueda(arr, medio + 1, fin, x);
}
```

- Dado `arr = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}` y buscando `x = 23`. Indique la secuencia de valores que toma la variable `medio` hasta encontrar el valor.
- ¿Qué ocurre si el array no está ordenado previamente?
- ¿Cuál es la complejidad del algoritmo en notación $\mathcal{O}$? Justifique basándose en cómo se reduce el tamaño del problema en cada llamada recursiva.

3.5. Bucles anidados (II)

Examine el siguiente fragmento de código:

```
public static void metodoLog(int n) {
    int a = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 1; j < n; j *= 2) {
            a++;
        }
    }
}
```

- Si `n = 10`, ¿cuántas veces se ejecuta la línea `a++` aproximadamente?
- ¿Qué relación matemática tiene la variable índice `j` con `n` en el bucle interior?
- Calcule la complejidad del algoritmo en notación $\mathcal{O}$. Justifique su respuesta.

3.6. *Bubble sort* optimizado

Estudie este código, una variante de un algoritmo de ordenación popular (*Bubble sort*):

```
void ordenar(int[] arr) {
    int n = arr.length;
    boolean intercambiado;
    for (int i = 0; i < n - 1; i++) {
        intercambiado = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // swap temp
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                intercambiado = true;
            }
        }
        if (intercambiado == false) break;
    }
}
```

- Si el array de entrada ya está ordenado (`{1, 2, 3, 4, 5}`), ¿cuántas veces se ejecuta la comparación `arr[j] > arr[j+1]`?
- ¿Para qué sirve la variable `intercambiado`?
- Indique la complejidad en el mejor caso y en el peor caso. ¿Por qué son diferentes?

3.7. Complejidad de un árbol N-ario

Dado una estructura de árbol N-ario similar a la del ejercicio 2.1 (clase con una lista de hijos), ¿cuál sería la complejidad temporal de un método recursivo que recorre el árbol completo para contar el número total de nodos, si `N` es el número total de nodos en el árbol?

3.8. Método misterioso (II)

Analiza el siguiente método recursivo que trabaja sobre un árbol binario. ¿Qué calcula exactamente?

```
public int metodoMisterioso(Node nodo) {
    if (nodo == null) return 0;
    int izq = metodoMisterioso(nodo.left);
    int der = metodoMisterioso(nodo.right);
    return 1 + Math.max(izq, der);
}
```

3.9. Árbol binario degenerado

Un árbol binario es degenerado (no piense mal) cuando todos los nodos tienen un único hijo siempre en el mismo lado (por ejemplo, el derecho). En este caso, ¿cuál es la complejidad computacional en notación $\mathcal{O}$ de una búsqueda binaria sobre este árbol, en el peor de los casos? ¿Y en el mejor?

3.10. Complejidad de método recursivo

Dado el siguiente método:

```
public void metodoRaro(int n) {
    if (n <= 0) return;
    System.out.println(n);
    metodoRaro(n - 1);
    metodoRaro(n - 1);
}
```

- ¿Cuántas llamadas a `metodoRaro` se hacen para $n=3$?
- ¿Cuál es la complejidad en notación $\mathcal{O}$ en función de n ? **Pista:** Fíjese en que cada paso duplica el trabajo.